

Code Architecture

Obstacle Challenge

For the Obstacle Challenge, the robot will primarily rely on the camera for navigation, with the other sensors serving specific supporting roles.

Sensor Reliance

Sensor	Approx. Reliance	Primary Purpose
Raspberry Pi Camera	75%	Obstacle detection, colour identification, navigation
TOF Sensors	5%	Parking, crash prevention, and final positioning
IMU + Rotary Encoder	Supporting	Parking guidance and path mapping

The TOF sensors will have the highest authority when detecting an immediate risk of collision. The IMU and rotary encoder may also be used after the first round to plot a stable path for the following two rounds.

Code Structure

Camera Computation & Processing:

The manner in which we process frames yielded by the camera is imperative to the functioning of our creation. Due to severe, unforeseen hardware limitations created due to the unavailability of the successor of the raspberry pi zero w, which would've provided us with a 64 bit multi-core processor with 5 times the processing power, we have been forced to optimise our computation process as much as possible to the extent of our ability, regardless if readability is compromised or not. The overall structure of our processing is as follows:

After initialization of the camera:

1. Raw 422 camera data is immediately captured in a "While (True)" loop at QQVGA, or 160x120 px.

2. We slice the raw array yielded by the camera using numpy to only obtain U and V data in the format of an array
3. The UV data is routed through a filter so that only pixels that match either red, green, purple, or grey are considered, while the rest is discarded (similar to a colour mask)
4. All x and y positions of the pixels that match the colour are stored in separate arrays, as well as a variable. In this variable, 0 = Red and 1 = Green. The code for detecting grey and purple will be separate.
5. The mean values of all x and y coordinates will be extracted, yielding the center x and y values respectively. This value will be used to find the relative angle of the obstacle on the screen. Initially, our approach was to find the corners of the obstacle, then run a pure pursuit algorithm, but that algorithm is much more prone to noise and failure, unreliable, and painful to implement. Our new approach is less prone to failure due to it taking the mean values of all values.

For the obstacle challenge:

6. The angle between the obstacle and the bot will be adjusted so that it allows the bot to overtake the obstacle. As the bot gets closer to the obstacle and therefore the obstacles start appearing to occupy more of the screen, the angle between the obstacle and bot will get smaller proportionally.
7. The angle of the bot with the obstacle will be adjusted by the steering system, running a PID algorithm to prevent undershooting or overshooting.
8. Depending on the colour of the obstacle, the bot will adjust the angle between itself and the bot. Using the centre of the camera as a “normal” or a fixed reference, any angle to the left is negative while any angle to the right is positive. If an obstacle is for instance red and on the right, the algorithm will try to position the obstacle onto the left to overtake it from the right

Parking Manoeuvre — Preliminary

The following parking sequence is not yet final and will be refined during testing.

After completing the third lap:

1. The robot will immediately reverse.
2. It will position itself at a distance of **X cm** from the wall.
3. **X** will be defined as:

$$X = k \times \text{parking-wall length}$$
 where **k** is a constant that will be determined through testing and tuning.
4. The robot will then drive alongside the parking area until the TOF sensor on the respective side briefly detects the parking wall.
5. The sensor, positioned approximately at the centre of the robot, will be aligned with the wall.

6. The robot will then use odometry to travel a distance **Y** forward. The value of **Y** will depend on the robot's turning characteristics and will be determined through testing.
7. The robot will reverse into the parking area.
8. Finally, the robot will perform one final turn, assisted by the rear TOF sensor, to align itself parallel to the parking wall.

Open Challenge

The code for the Open Challenge will be considerably simpler.

The robot will primarily:

- Use the camera to determine when turns are required.
- Use the TOF sensors to regulate its position relative to the track and surrounding walls.
- Operate the drive motors at their maximum practical speed.
- Complete parking by stopping a few seconds after crossing the second-last turn of the third lap.

The Open Challenge strategy will therefore prioritize simple camera-based navigation, positional correction through the TOF sensors, and maximum practical motor performance.